

中国科学院大学
University of Chinese Academy of Sciences

操作系统
课程简介

中国科学院大学计算机学院

2026-08-31



内容提要

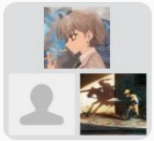
- 课程信息
- 什么是系统?
- 什么是操作系统?
- 怎么学习操作系统?

课程信息

- 课程网站： https://mooc.ucas.edu.cn/
- 授课教师：
  - 蒋德钧 中科院计算所 jiangdejun@ict.ac.cn
- 助教：
  - 余俊达 yujunda20@mails.ucas.ac.cn
  - 刘爽 liushuang226@mails.ucas.ac.cn
- 同期课程：
  - 操作系统2班：包云岗
  - 操作系统研讨课：蒋德钧

课程信息

- 微信群



群聊：操作系统1班课程群2026秋

该二维码7天内（9月6日前）有效

课程教材

- 课程教材
  - 现代操作系统（第4或5版）Andrew S. Tanenbaum，Herbert Bos 著
    - 中文版：陈向群、马洪兵等译，机械工业出版社 ISBN: 9787111573692（第4版）/ 9787111776895（第5版）
- 参考教材
  - Operating Systems: Three Easy Pieces, Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau

    https://pages.cs.wisc.edu/~remzi/OSTEP/
  - 操作系统：原理与实现；陈海波、夏虞斌等著，机械工业出版社；ISBN：9787111722489



实例分析

- 内容
  - XV6操作系统：一个类UNIX的教学操作系统
  - 通过阅读XV6操作系统代码，加深对操作系统原理和设计的理解
- 课时安排
  - 共4次，每次2个课时
  - 2课时详细分析XV6代码
    - 课前分组阅读代码
    - 课上按小组解读对代码的理解
  - 另根据课时进度穿插介绍操作系统前沿研究

实例分析

- 分组方式
  - 约3人一组，自由组合
- 抽签方式
  - 每次抽4~5个组
  - 提前一周抽组号以及各组负责主讲的部分
  - 提前2~3天抽组内主讲人
    - 主讲人在课上介绍自己组阅读的代码部分
    - 其他同学辅助介绍
    - 遇到生病或者临时有事无法讲的情况，需要提前告诉助教



实例分析

- XV6：面向RISC-V用C语言实现的Unix like OS
  - https://pdos.csail.mit.edu/6.828/2026/xv6.html
- Linux 源码（可选择5.10或以上版本）
  - http://cdn.kernel.org/
- 阅读部分
  - （1）进程与线程管理
  - （2）并发、同步和进程通信机制
  - （3）虚拟内存
  - （4）文件系统
  - XV6和Linux源码相结合

上课时间

- 教室：教三楼107
- 时间： 每周三 + 单周的周一
  - 5-6节课：13:30 – 14:15, 14:20 – 15:05
  - 调补课或特殊情况会提前通知

年度	2026																	2027	
月份	九月				十月					十一月				十二月				一月	
教学 周次	教学周																	考试周	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
星期一	31老生开课	7 新生开课	14	21	28	5	12	19	26	2	9	16	23	30	7	14	21	28	4	11
星期二	1	8	15	22	29	6	13	20	27	3	10	17	24	1	8	15	22	29	5	12
星期三	2	9	16	23	30	7	14	21	28	4	11	18	25	2	9	16	23	30	6	13
星期四	3	10	17	24	1国庆节	8	15	22	29	5	12	19	26	3	10	17	24	31	7	14
星期五	4	11	18	25中秋节	2	9	16	23	30	6	13	20	27	4	11	18	25	1元旦节	8	15
星期六	5	12	19	26	3	10补周三课	17	24	31	7	14	21	28	5	12	19	26	2	9	16
星期日	6	13	20补周二课	27	4	11	18	25	1	8	15	22	29	6	13	20	27	3	10	17
说 明	1.秋季学期从2026年8月31日开始； 2.2027年1月17日课程结束；																			
	3.2027年1月18日—2027年2月21日学生放寒假五周； 4.2027年2月22日春季学期正式开课。																			
	注：教职工放假由党政办统一安排。																			

考核方法

- 平时作业：20%
- 实例分析：20%
- 期中考试：20%
- 期末考试：40%

前序课程

- 前序基础课程
  - 程序设计（C语言）
  - 数据结构
  - 计算机组成原理
  - 汇编语言（代码实例分析会用到）



内容提要

- 课程信息
- 什么是系统?
- 什么是操作系统?
- 怎么学习操作系统?

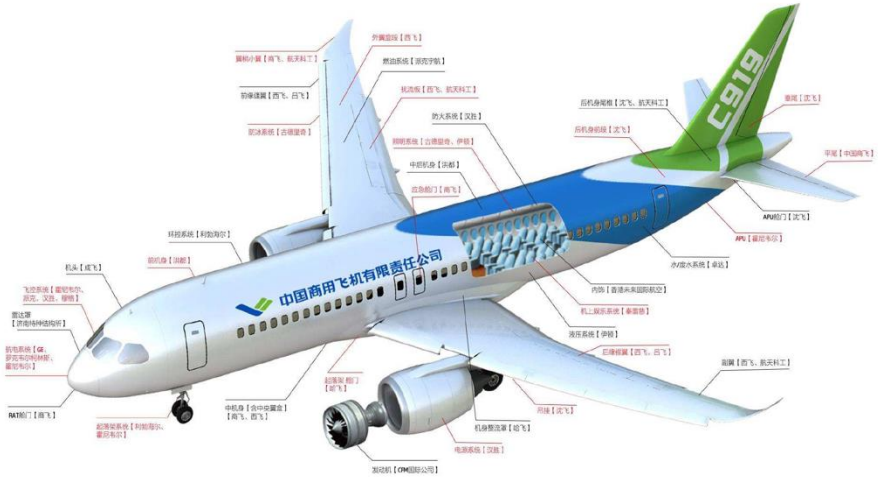
系统（System）

- 由一群有关联的组件/部件/模块/个体组成，根据某种规则协同运作，能完成单个组件不能单独完成的工作的群体/整体

组件



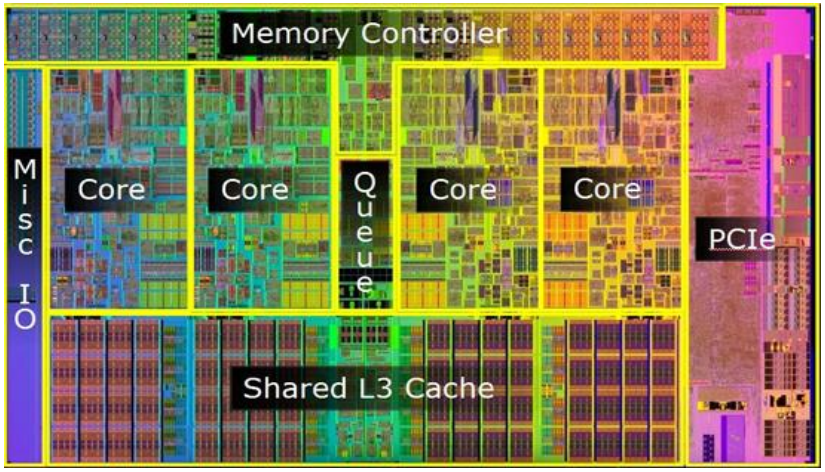
系统

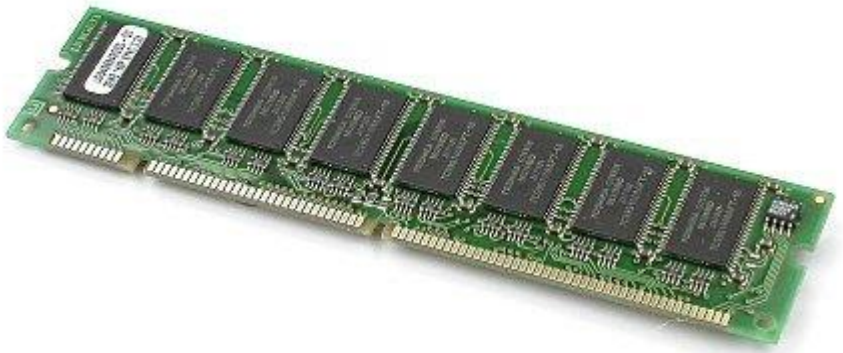




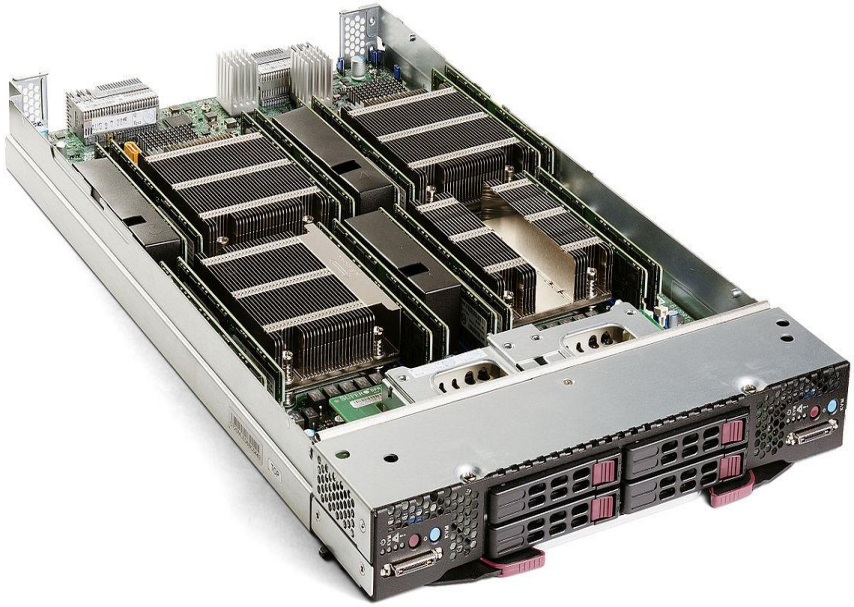


计算机系统（硬件）









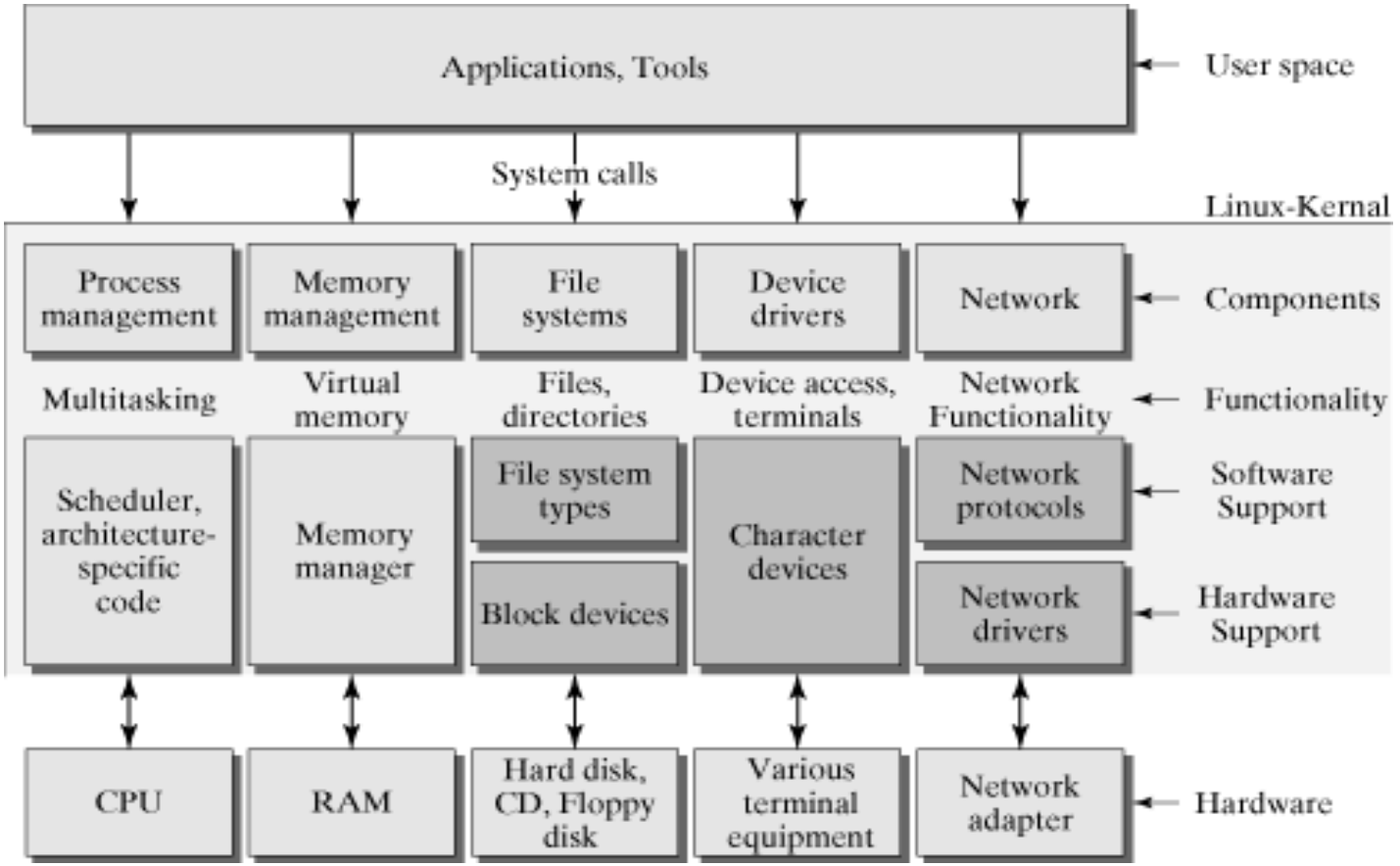
[图片来源: Wikipedia]

计算机系统（硬件）



[图片来源: Baidu]

计算机系统（软件）



[图片来源：linux-india]

系统的特征

- 系统是复杂的
  - 不同组件相互交互
  - 每一个组件自身也很复杂
- 系统设计是多目标的
  - 功能
  - 性能
  - 规模
  - 可靠性
  - 成本
  - 安全
  - …

系统的特征

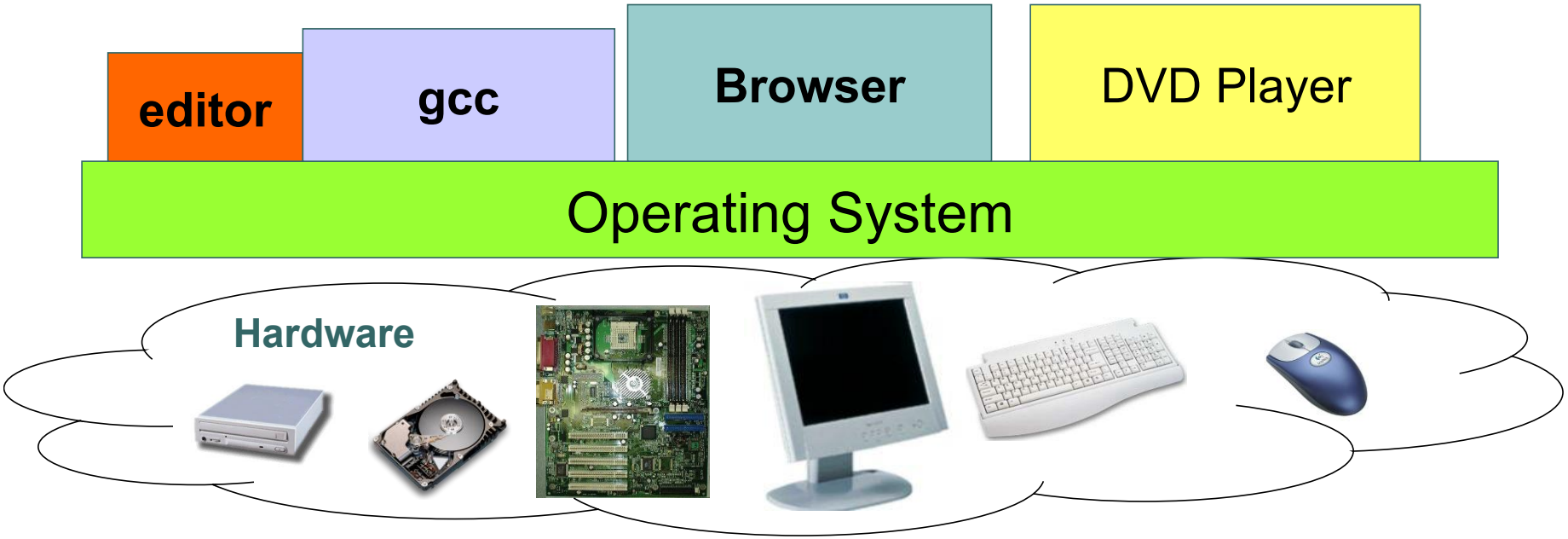
- 系统工作 vs. 理论工作
  - 理论工作：寻找极限（lower bound, upper bound）
    - 数学“孪生素数猜想”（张益唐，2013）：是否存在孪生素数间最大间隔的常数
    - 物理学“标准模型”：描述强力、弱力及电磁力这三种基本力及组成所有物质的基本粒子的理论
  - 系统工作：追求权衡（trade off）
    - 缓存：空间和时间的权衡
    - 日志：性能和可靠性的权衡
    - 在实际实现系统前，明确你的系统设计目标

内容提要

- 课程信息
- 什么是系统?
- 什么是操作系统?
- 怎么学习操作系统?

什么是操作系统?

- 承上启下
  - 在应用和硬件之间的一层软件
  - 对上层软件提供硬件抽象、实现共用功能和操作接口
  - 对底层硬件进行管理、实现共享、保证隔离





操作系统能做什么？

- 一个程序的运行

```
#include <stdio.h>

int main(int argc, char *argv[]) {
   int num = 0, base = 3;
   int result = 0;

   printf("Please input a number: ");
   scanf("%d", &num);

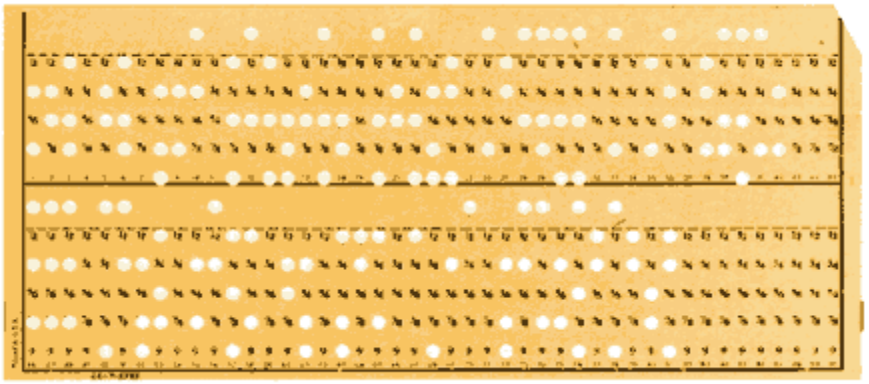
   result = base + num;
   printf("Hello %d\n", result);

   return 0;
}
```

操作系统能做什么?

- 抽象与接口
  - 让硬件更方便使用
    - 接收输入: scanf
    - 计算：使用CPU
    - 产生输出: printf
  - 让软件更容易编写
    - 不用每个程序自己编写输入、输出函数

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```





操作系统能做什么?

- 多个程序的运行

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
   char *str = argv[1];

   for(int i=0;i<3;i++) {
      printf("Hello %s\n", str);
      sleep(1);
   }

   return 0;
}
```

```
./a.out U & ./a.out C & ./a.out A
& ./a.out S &

Hello U
Hello C
Hello A
Hello S
Hello U
Hello S
Hello A
Hello C
Hello A
Hello C
Hello S
Hello U
```



操作系统能做什么？

- CPU资源使用、管理与隔离
  - 一个CPU core，多个程序同时运行
    - 让多个程序共享使用一个CPU core
    - 分配CPU资源，调度程序执行
    - 有程序需要长时间使用CPU，是否允许？
    - 有程序需要优先使用CPU，是否允许？
    - ...
  - 多个CPU cores，多个程序同时运行
    - 分配CPU资源，调度程序执行
    - 是否允许一个程序独占一个CPU core？
    - …



操作系统能做什么?

- 一个程序的运行

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int *array = malloc(5*sizeof(int));
    printf("The starting address of the in-memory array is %p\n", array);

    array[0] = 1;
    for(int i=1;i<5;i++) {
      array[i] = array[i-1] + i;
      printf("The %d element is %d\n", i, array[i]);
    }

    return 0;
}
```



操作系统能做什么?

- 一个程序的运行

./a.out & ./a.out &

The starting address of the in-memory array is 0x7fc83c4027d0
The starting address of the in-memory array is 0x7f87924027d0

The 1 element is 2
The 2 element is 4
The 3 element is 7
The 1 element is 2
The 4 element is 11
The 2 element is 4
The 3 element is 7
The 4 element is 11



操作系统能做什么？

- 内存资源使用、管理与隔离
  - 给程序分配内存资源
  - 让程序读写内存（提供寻址机制）
  - 程序结束后回收内存资源
  - 不同程序共享使用内存
    - 是否允许不同程序访问同一个内存地址？
    - 程序A能否访问程序B的内存数据？
    - 内存不够用时，如何处理？
    - …



操作系统能做什么？

- 一个程序的运行

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <fcntl.h>

int main(int argc, char *argv[]) {
   int fd = open("/tmp/examplefile",
O_CREAT|O_WRONLY, S_IRWXU);

   int rc = write(fd, "hello world\n", 13);

   close(fd);
   return 0;
}
```

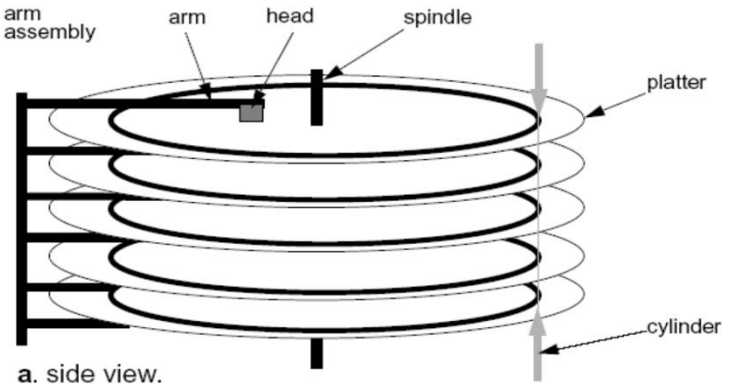
```
./a.out

cat /tmp/examplefile
hello world
```

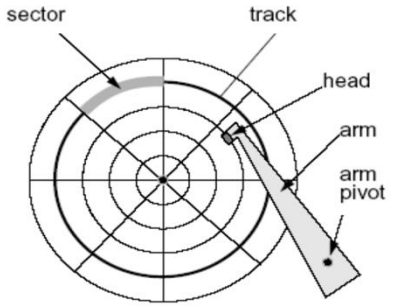
操作系统能做什么？

- 数据持久化与存储设备管理
  - 让程序能在持久化存储设备上读写数据
  - 数据安全保护、数据可靠性保护
  - 和不同的存储设备交互，驱动管理
  - 屏蔽存储设备硬件细节，同时管理存储设备





a. side view.



b. top view.



操作系统能做什么？

- 一个程序的运行

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;
int loops;

void *worker(void *arg) {
   for (int i=0;i<loops;i++) {
      counter++;
   }
   return NULL;
}
```

```
int main(int argc, char *argv[]) {
   loops = atoi(argv[1]);
   printf("Initial value: %d\n", counter);

   pthread_t p1, p2;

   pthread_create(&p1, NULL, worker,NULL);
   pthread_create(&p2, NULL, worker,NULL);

   pthread_join(p1, NULL);
   pthread_join(p2, NULL);

   printf("Final value: %d\n", counter);

   return 0;
}
```



操作系统能做什么?

- 并发访问控制
  - 控制多个程序访问相同资源时的正确性
    - 如何保证程序操作的原子性?
    - 如何让其他程序知道当前程序操作已完成?
  - 再思考
    - 能否把程序访问变成串行执行? 串行执行的粒度多大?

```
./a.out 1000
Initial value: 0
Final value: 2000

./a.out 10000
Initial value: 0
Final value: 20000

./a.out 100000
Initial value: 0
Final value: 115664
```



操作系统能做什么？

- 还有什么？
  - 不同外部设备的管理
  - 安全保护
    - 一个用户程序能否执行任意指令
    - 一个用户程序崩溃不能引起整个系统崩溃
  - 数据如何传输给远程计算机
  - 虚拟化：虚机、容器
  - AI应用

操作系统的发展

- IBM 701计算机 1954
  - OS是一个通用函数库
  - 用户自行提交程序，每个用户被分配一段机时。机时用完，则换另一个用户的程序使用
- IBM 709计算机 1959
  - 支持批处理：使用一个管理程序自动切换不同的作业
  - 单道批处理系统
    - 一次只有一个作业在内存中运行

操作系统的发展

- 1960s
  - 多道批处理系统（MultiProgramming）
    - 支持多个程序同时在内存中运行
  - 需要支持CPU、内存的共享使用，支持不同程序的切换
  - 批处理形式运行，程序运行直到主动放弃CPU（完成或处理I/O请求）
  - 以提升资源利用率（CPU）为目标
- Multics 1965、UNIX 1974
  - 分时操作系统（Time-sharing Operating System）
  - 提供时间片，不同程序基于时间片切换
  - 支持人机交互、计算机资源共享与隔离

操作系统的发展

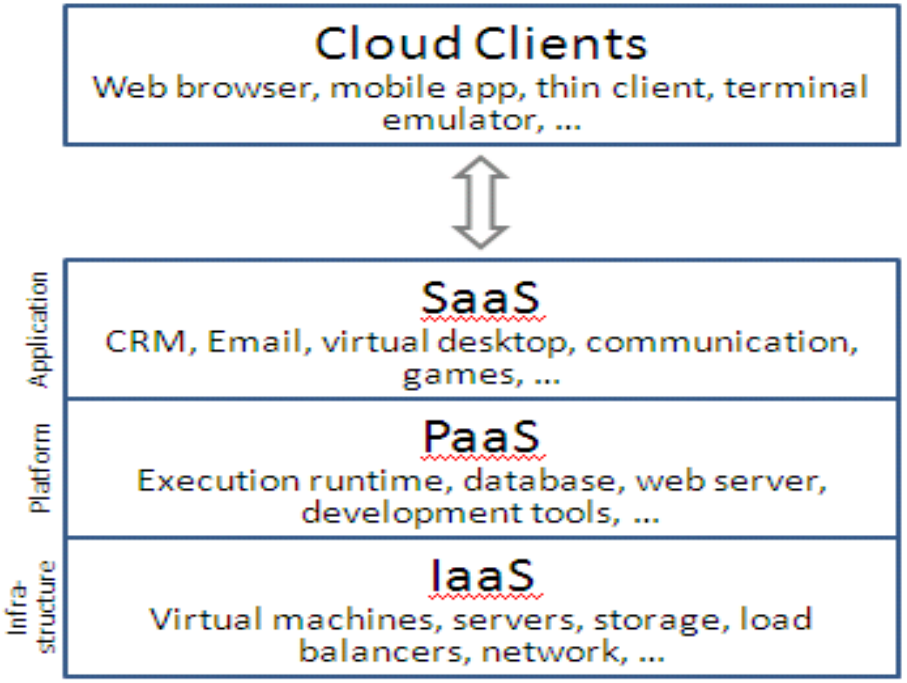
- Alto operating system
  - 64KB内存，2.5MB磁盘
  - 图形显示
  - 以太网
  - 文件系统
  - 顺序执行的单用户系统
- DOS
- Windows
- Mac OS
- Linux：RedHat、Ubuntu、OpenEuler



第一台PC Alto(Xerox PARC)

操作系统的发展

- 集群
  - 网络连接的计算机
  - 商用操作系统 + 分布式框架（如MapReduce，Spark）
- 云计算
  - 数据中心是一个计算机
  - 按需使用资源
  - 数据中心操作系统
    - Mesos
    - Kubernetes



操作系统的发展

- 物联网
  - 万亿规模的物端设备
  - 物端应用需求差异化
  - 嵌入式OS
    - RT-Thread, LiteOS
  - 手机OS
    - iOS, Android
  - 物端操作系统：鸿蒙







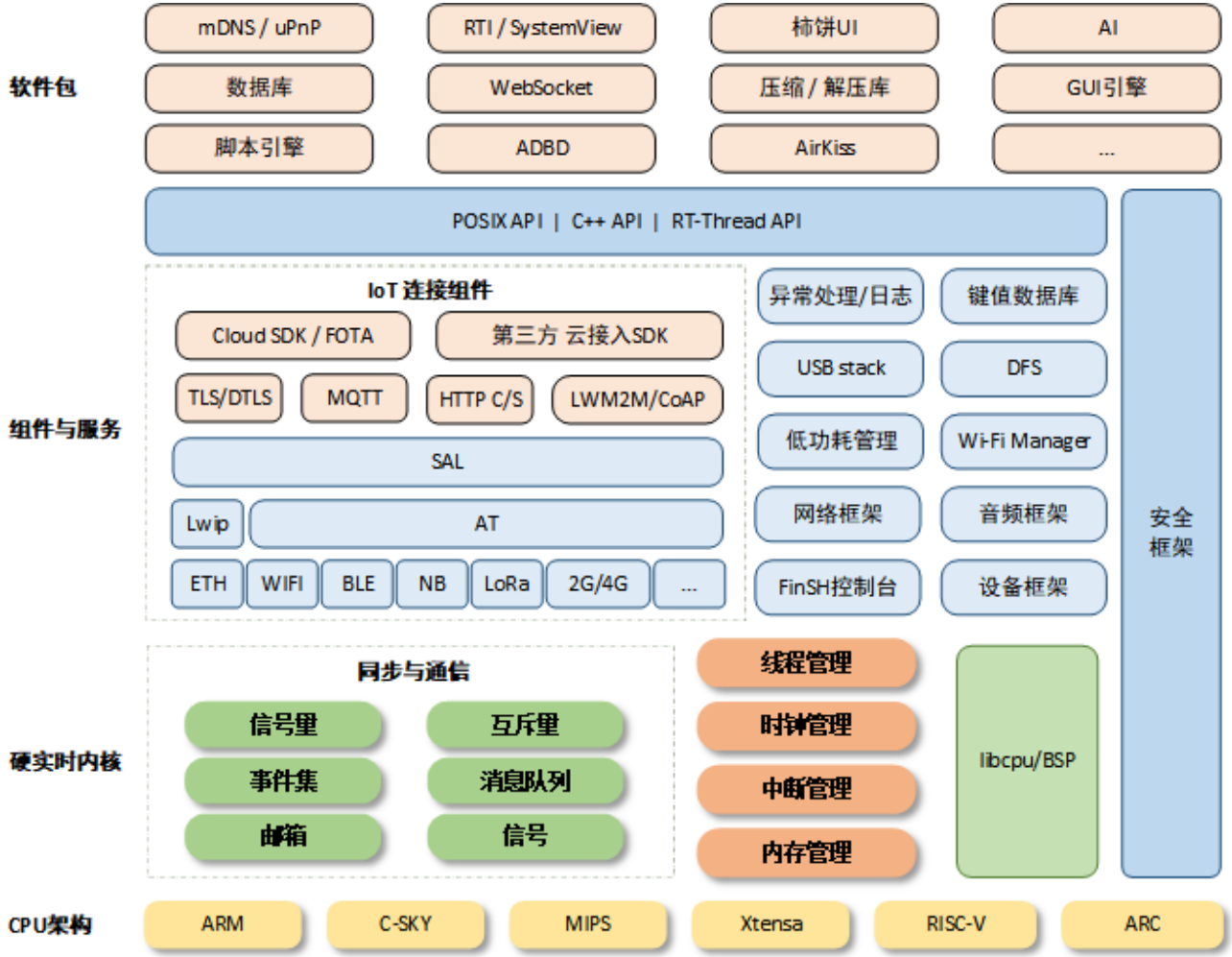






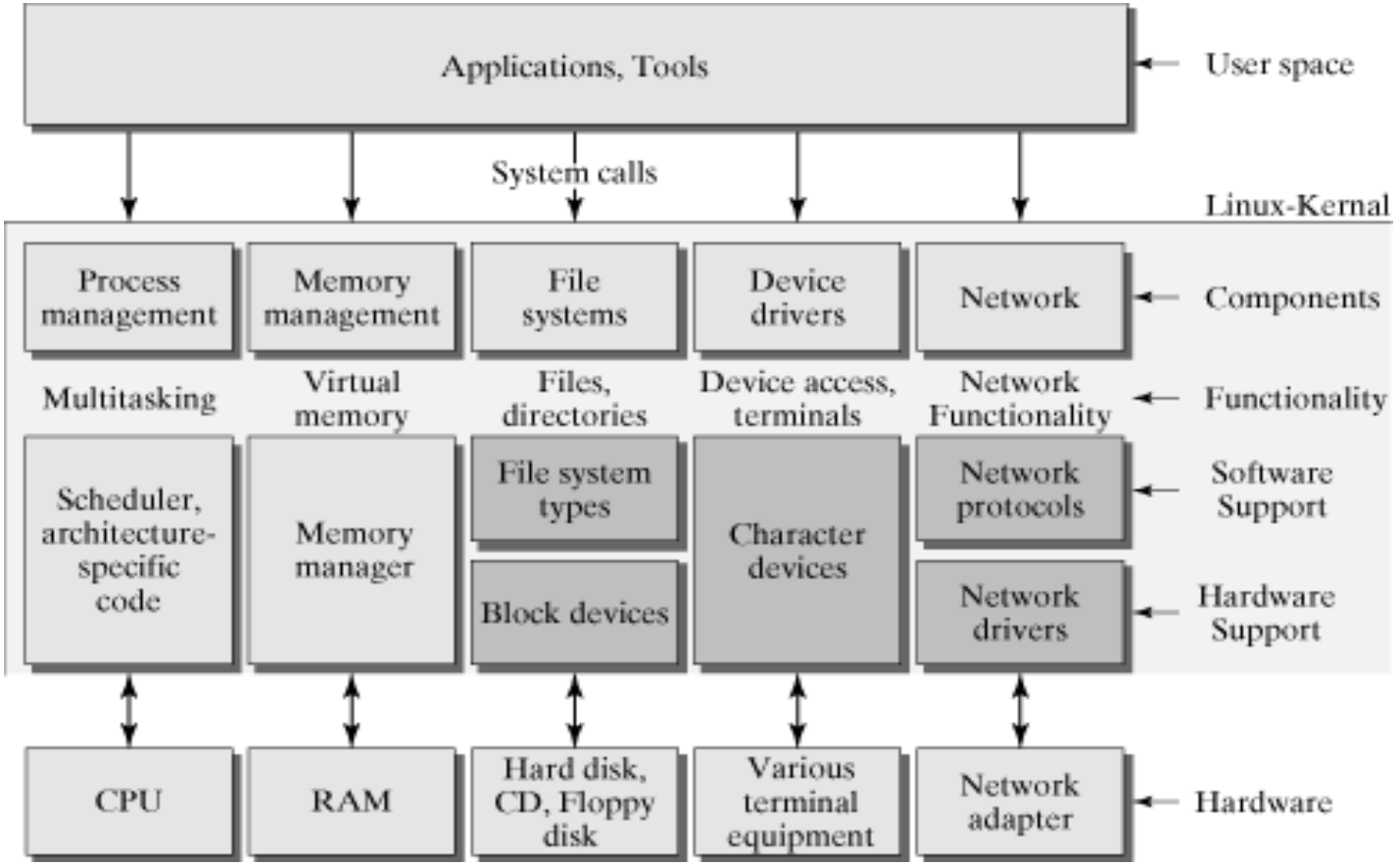
操作系统示例

- 示例：RT-Thread



操作系统示例

- 对比：Linux vs. RT-Thread



[图片来源：linux-india]



内容提要

- 课程信息
- 什么是系统?
- 什么是操作系统?
- 怎么学习操作系统?

学习目标

- 了解计算机操作系统发展
- 掌握操作系统的基本概念与主要模块（What）
  - OS是计算机系统的核心部分
  - 很多研究领域的基础：网络、存储系统、数据库、分布式系统等
- 熟悉OS设计原理与关键机制（How & Why）
  - 理解操作系统如何工作
  - 理解操作系统为什么这么设计
  - OS设计思想可以用于其他系统或领域
- 培养系统观
  - OS的设计选择体现系统观
    - 并发控制、页表、缓存等

怎么学习操作系统?

- 操作系统很复杂，要考虑的东西真多，我很喜欢!
- 操作系统很复杂，要考虑的东西真多，我不喜欢!
- “我曾经给本科生教操作系统课，几个星期下来，有的学生就对系统方向的研究上瘾了，觉得特有意思，而另一批学生就特别反感，认为系统过于复杂，需要考虑的东西太多。”



周源源教授 UCSD



怎么学习操作系统?

- 一个转变
  - 从“用操作系统”到“设计操作系统”
  - 思路：该如何管理应用程序和硬件资源
- 两个要求
  - 认真听讲：有问题欢迎随时提问
  - 认真完成作业：有问题非常欢迎讨论



怎么学习操作系统?

- “理论联系实际”
  - 操作系统是一门实践性很强的课程
- 阅读代码
  - 阅读并理解XV6操作系统代码
    - 结构、流程
    - 设计思路
- 开发代码
  - 实现一个小的操作系统（操作系统研讨课）
  - 在Linux上自行开发一些程序
    - 多线程、并发控制
    - 系统调用
    - 内核模块、设备驱动



怎么学习操作系统?

- 和操作系统相关的学术会议
  - SOSP，OSDI，FAST，EuroSys
  - HotOS：crazy idea for operating system

怎么学习操作系统?

- 操作系统仍然在变化
  - 底层硬件不断在变化
    - CPU、GPU、XPU、内存、存储、网络
  - 上层应用需求不断在变化
    - AI、云计算、大数据、边缘计算
  - 大模型与AI agent的影响
    - 大模型参与系统决策 （AI for OS）
    - Agent成为OS的组件



怎么学习操作系统?

- 操作系统仍然在变化
  - 操作系统的结构也随之相应变化
  - 设计边界不再严格
    - 应用、库函数、内核模块
    - 用户态、内核态
    - 本地资源、远程资源
  - 设计原则没有变
    - 针对应用场景需求做出合适的选择
    - Trade off



怎么学习操作系统?

- 关于AI
  - 要不要用AI?
    - 要
  - 基本原则是什么?
    - 让AI辅助你，而不是让AI代替你
  - OS课的特点
    - OS课需要你深度思考、理解背后的原因和机制
    - 难于及时反馈价值，是一门延迟满足的课程
    - AI可以给你讲述概念、帮你做作业，但这是AI“懂了”，并不是你掌握了
  - 最后一个问题
    - 你如何验证AI给出的内容、代码?
  - 还有一点：AI费Token ☹



课程内容安排

- 讲授内容：
  - 操作系统结构
  - CPU管理：进程、线程和中断
  - 并发控制：同步、进程间通信
  - 内存管理：虚存管理
  - 设备管理：驱动、磁盘、固态盘
  - 存储管理：文件系统
- 实例分析：
  - XV6 实例分析（报告和讨论）



总结

- 系统概念与特征
  - 多个组件，相互交互，完成单个组件无法完成的功能
  - 多目标设计与平衡
- 操作系统能做什么
  - 承上启下：管理硬件资源 + 给应用提供抽象和接口
    - CPU、内存、存储、网络、设备管理
    - 并发控制
    - 安全防护
- 如何学习操作系统
  - “用操作系统” → “设计操作系统”
  - 理论联系实践